

03-typescript

03. TypeScript Basics

Level: □ Beginner

Jam: 4 (2 minggu × 2 sesi)

Prasyarat: JavaScript Fundamentals

Output: Project JavaScript di-refactor ke TypeScript

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Paham kenapa TypeScript dan bedanya sama JavaScript
- Nulis tipe data dasar (string, number, boolean, union, literal)
- Bikin interface & type alias buat object
- Nulis function pake tipe, generic, dan overload
- Konfigurasi tsconfig.json dan compile TypeScript

Materi

Sesi	Topik	File
1	Type basics — tipe data, type inference, union, literal types	01-basics.
2	Types & Interfaces — object types, interface vs type, optional, readonly	02-types-i
3	Functions & Generics — function types, overload, generics, constraints	03-functio
4	tsconfig & Project — tsconfig.json, strict mode, compiling, project references	04-tsconfig

Output Akhir Modul

Refactor Project — ambil project JavaScript minggu sebelumnya, tambahin tipe TypeScript, interface, generic function. Compile & jalanin tanpa error.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain this type error and how to fix it"
- "Convert this JavaScript function to TypeScript with proper types"
- "Generate an interface for this API response"
- "Refactor this code to use generics"

3.1 Tipe Dasar TypeScript

Kenapa TypeScript?

TypeScript nambahin **static types** ke JavaScript. Error ketangkap pas ngetik, bukan pas jalan.

// JavaScript – error baru kelihatan pas runtime

```
function add(a, b) { return a + b; }  
console.log(add("5", 10)); // "510" – bug!
```

// TypeScript – error langsung kelihatan di editor

```
function add(a: number, b: number): number { return a + b; }  
console.log(add("5", 10)); // ERROR: Argument of type 'string' not assignable
```

Tipe Data Dasar

```
let nama: string = "Budi";  
let umur: number = 17;  
let isActive: boolean = true;  
let sesuatu: any = "bebas"; // Hindari – matikan type checking  
let kosong: null = null;  
let belumDiisi: undefined = undefined;
```

Type Inference

TypeScript otomatis nebak tipe dari value.

```
let nama = "Budi"; // TypeScript tau ini string  
let umur = 17; // number  
let isActive = true; // boolean
```

```
// nama = 42; // ERROR: Type 'number' not assignable to type 'string'
```

Array & Tuple

```
// Array
let angka: number[] = [1, 2, 3];
let nama: Array<string> = ["Budi", "Andi"]; // syntax alternatif

// Tuple – array dengan panjang & tipe tetap
let siswa: [string, number] = ["Budi", 17];
// siswa = [17, "Budi"]; // ERROR: urutan salah
```

Union Types

Variable bisa punya lebih dari satu tipe.

```
let id: string | number;
id = "ABC123"; // OK
id = 123;      // OK
// id = true;  // ERROR

function display(value: string | number): string {
  return `Value: ${value}`;
}
```

Literal Types

Batasin value ke nilai tertentu, bukan cuma tipe.

```
let warna: "merah" | "hijau" | "biru";
warna = "merah"; // OK
// warna = "kuning"; // ERROR

type Status = "pending" | "success" | "error";
let status: Status = "pending";
```

Type Aliases

Bikin nama kustom buat tipe.

```
type ID = string | number;
type Point = { x: number; y: number };

let userId: ID = "user_001";
let pos: Point = { x: 10, y: 20 };
```

Latihan

1. Bikin variable dengan tipe string, number, boolean, dan array of numbers. Coba assign value yang salah — lihat error TypeScript
2. Bikin fungsi formatInput yang nerima string | number dan return string berisi "Input: <value>"
3. Bikin type alias UserId (string | number) dan StudentStatus ("active" | "inactive" | "graduated")
4. Bikin tuple [string, number, boolean] buat representasi data siswa (nama, umur, isActive). Coba akses tiap element dan lihat tipenya

3.2 Object Types & Interfaces

Object Types

Cara ngetik object di TypeScript.

```
let siswa: { name: string; age: number } = {  
  name: "Budi",  
  age: 17,  
};
```

Interface

Interface adalah cara utama buat describe object shape.

```
interface Student {  
  name: string;  
  age: number;  
  grade?: string; // optional  
}  
  
const budi: Student = { name: "Budi", age: 17 };  
  
// Extended interface  
interface Graduate extends Student {  
  graduationYear: number;  
}  
  
const andi: Graduate = {  
  name: "Andi",  
  age: 22,  
  graduationYear: 2025,  
};
```

Type Alias vs Interface

Keduanya mirip, ada beda tipis.

```
// Type alias – bisa buat union, tuple, primitive
type ID = string | number;
type Point = { x: number; y: number };

// Interface – lebih cocok buat object, bisa declaration merging
interface Car {
  brand: string;
  model: string;
}

// Declaration merging (interface doang)
interface Car {
  year: number;
}
// Sekarang Car punya brand + model + year
```

Table bedanya:

Fitur	Interface	Type Alias
Object shape	□ Ya	□ Ya
Declaration merging	□ Bisa	□ Ga bisa
Union / Tuple	□	□
Extends	extends	Intersection (&)
Computed properties	□	□

Optional & Readonly

```
interface Config {
  readonly apiKey: string; // gabisa diubah setelah inisialisasi
  port?: number;           // optional – boleh ga diisi
  debug: boolean;
}

const config: Config = {
  apiKey: "abc123",
  debug: true,
  // port optional, ga harus diisi
};

// config.apiKey = "newkey"; // ERROR: readonly
```

Intersection Types

Gabungkan beberapa tipe pake &.

```
interface Person {
  name: string;
  age: number;
}

interface Employee {
  employeeId: string;
  department: string;
}

type WorkerPerson = Person & Employee;

const worker: WorkerPerson = {
  name: "Budi",
  age: 17,
  employeeId: "EMP001",
  department: "Engineering",
};
```

Nested Objects

```
interface Address {
  street: string;
  city: string;
  postalCode: string;
}

interface StudentWithAddress {
  name: string;
  age: number;
  address: Address;
}

const student: StudentWithAddress = {
  name: "Budi",
  age: 17,
  address: {
    street: "Jl. Merdeka No. 1",
    city: "Jakarta",
    postalCode: "10110",
  },
};
```

Latihan

1. Bikin interface Product dengan properti: id (string), name (string), price (number), category (optional string). Bikin 2 object Product
2. Bikin interface CartItem yang punya product (Product), quantity (number). Bikin interface ShoppingCart yang punya items (CartItem[]) dan totalPrice (number)
3. Bikin type Shape yang union dari Circle (radius: number) dan Rectangle (width: number, height: number) pake intersection & discriminated union
4. Bikin 2 interface Teacher dan Student, terus bikin intersection type Mentor dari keduanya. Tambah properti mentorSince: Date

3.3 Functions & Generics

Function Types

Nulis tipe buat parameter dan return value.

```
// Basic function dengan tipe
function add(a: number, b: number): number {
    return a + b;
}

// Arrow function
const multiply = (a: number, b: number): number => {
    return a * b;
};

// Void – fungsi yang ga return apa-apa
function logMessage(msg: string): void {
    console.log(msg);
}
```

Optional & Default Parameters

```
// Optional parameter
function greet(name: string, greeting?: string): string {
    return `${greeting ?? "Halo"}, ${name}!`;
}

// Default parameter
function createEmail(name: string, domain: string = "gmail.com"): string {
```

```

    return `${name}@${domain}`;
}

console.log(createEmail("budi")); // budi@gmail.com
console.log(createEmail("budi", "yahoo.com")); // budi@yahoo.com

```

Function Type Signatures

Simpen tipe fungsi di variable atau parameter.

```

// Type buat fungsi
type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (x, y) => x + y;
const subtract: MathOperation = (x, y) => x - y;

// Fungsi yang terima fungsi lain (callback)
function calculate(a: number, b: number, op: MathOperation): number {
    return op(a, b);
}

console.log(calculate(10, 5, add)); // 15
console.log(calculate(10, 5, subtract)); // 5

```

Overload

Bikin beberapa signature buat fungsi yang sama.

```

function process(value: string): string[];
function process(value: number): number[];
function process(value: string | number): string[] | number[] {
    if (typeof value === "string") {
        return value.split("");
    }
    return Array.from({ length: value }, (_, i) => i + 1);
}

console.log(process("abc")); // ["a", "b", "c"]
console.log(process(5)); // [1, 2, 3, 4, 5]

```

Generics

Fungsi yang work dengan berbagai tipe tanpa kehilangan type safety.

```

// Generic function
function firstElement<T>(arr: T[]): T | undefined {

```



```

    return arr[0];
}

const num = firstElement([1, 2, 3]);    // number
const str = firstElement(["a", "b"]);    // string
// TypeScript tau return type sesuai input

```

Multiple Generic Types

```

function pair<A, B>(a: A, b: B): [A, B] {
    return [a, b];
}

const result = pair("Budi", 17); // [string, number]

```

Generic Constraints

Batasin generic pake extends.

```

interface HasLength {
    length: number;
}

function logLength<T extends HasLength>(item: T): T {
    console.log(`Length: ${item.length}`);
    return item;
}

logLength("Hello");    // OK – string punya length
logLength([1, 2, 3]);  // OK – array punya length
// logLength(123);      // ERROR – number ga punya length

```

Generic Interface

```

interface ApiResponse<T> {
    status: number;
    data: T;
    message: string;
}

// Pake interface generic
const userResponse: ApiResponse<{ id: string; name: string }> = {
    status: 200,
    data: { id: "1", name: "Budi" },
    message: "Success",
}

```

```
};

const listResponse: ApiResponse<string[]> = {
  status: 200,
  data: ["a", "b", "c"],
  message: "OK",
};
```

Latihan

1. Bikin fungsi `identity<T>` yang return value yang sama persis dengan tipenya. Test pake string dan number
2. Bikin fungsi `mergeObjects<T, U>` yang gabungin dua object. Return type harus intersection dari T & U
3. Bikin fungsi overloaded format — kalo nerima number return string dengan format "Rp ", kalo nerima Date return string "DD-MM-YYYY"
4. Bikin generic interface `Stack<T>` dengan method `push(item: T)`, `pop(): T | undefined`, `peek(): T | undefined`. Implementasiin simple array-based

3.4 tsconfig & Project Setup

Setup Project

Install TypeScript dan inisialisasi.

```
npm init -y
npm install --save-dev typescript
npx tsc --init
```

Ini bikin file `tsconfig.json` — otak dari project TypeScript.

tsconfig.json — Dasar

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  },
}
```

```

    "include": ["src"],
    "exclude": ["node_modules", "dist"]
  }

```

Penjelasan:

Option	Fungsi
target	Versi JS output (ES5, ES2020, ESNext)
module	Module system (commonjs, ES2020, ESNext)
outDir	Folder hasil compile
rootDir	Folder source code
strict	Aktifin semua type checking ketat
esModuleInterop	Biarin import dari CommonJS module

Strict Mode

strict: true ngaktifin beberapa aturan sekaligus.

// strictNullChecks – ga bisa assign null/undefined sembarangan

// Dengan strict:

```
let name: string = "Budi";
```

```
// name = null; // ERROR
```

// noImplicitAny – harus kasih tipe explicit

// function add(a, b) { return a + b; } // ERROR: parameter 'a' implicitly has '

// strictFunctionTypes – function parameter covariance check lebih ketat

Compiling

Compile sekali

```
npx tsc
```

Watch mode – compile otomatis tiap ada perubahan

```
npx tsc --watch
```

Compile specific file

```
npx tsc src/index.ts --outDir dist
```

Struktur Folder

```

project/
├── src/
│   └── index.ts

```

```

├── ┌── utils.ts
│   └── types.ts
├── dist/      # hasil compile – ga perlu di-commit
├── tsconfig.json
├── package.json
└── .gitignore

```

.gitignore

```

node_modules/
dist/
*.js
!*.config.js

```

Project References

Buat project besar, split jadi beberapa sub-project.

```

// root tsconfig.json
{
  "references": [
    { "path": "../shared" },
    { "path": "../frontend" },
    { "path": "../backend" }
  ]
}

```

Masing-masing sub-project punya tsconfig sendiri:

```

// shared/tsconfig.json
{
  "compilerOptions": {
    "composite": true,
    "outDir": "../dist",
    "rootDir": "../src"
  },
  "include": ["src"]
}

```

Type Declaration Files (.d.ts)

File .d.ts describes tipe buat JavaScript library yang ga punya type sendiri.

```

// myLib.d.ts
declare function myLib(message: string): void;
declare namespace myLib {

```

```
    function version(): string;
}

// Kalo library punya DefinitelyTyped:
// npm install --save-dev @types/nama-library
```

Debugging

```
# Generate source maps biar bisa debug TypeScript langsung
# tambah di tsconfig:
# "sourceMap": true

# Run langsung pake ts-node (ga perlu compile dulu)
npx ts-node src/index.ts

# Atau pake tsx (lebih cepet)
npx tsx src/index.ts
```

Latihan

1. Init project TypeScript dari 0: `npm init`, install `typescript`, `tsc` `--init`. Set `rootDir` ke `./src` dan `outDir` ke `./dist`. Bikin `src/index.ts` dengan `console log`, jalanin `npx tsc`, jalanin hasilnya pake `node`
2. Aktifin `strict: true`. Coba tulis kode yang error karena `strict` (`null assignment`, `implicit any`, dll). Fix error-nya
3. Bikin 3 file: `src/types.ts` (interface), `src/utils.ts` (generic function), `src/index.ts` (main). Import antar file. Compile dan jalanin
4. Install `@types/node`. Bikin script yang pake `process.argv` untuk baca CLI arguments. Compile dan jalanin dari terminal